

Secure Hospital Management System

Final Project Report

CYBS 3307 – Database Security
University of Hafr Al Batin
Term 2252

	Details
Project Objective	To develop a secure Electronic Health Record (EHR) database system that protects patient privacy, controls unauthorized access, and supports HIPAA-aligned security safeguards.
Prepared By	████████████████████ ████████████████████ ████████████████████ Raim bunaidar al-dhafiry ██████████
Submitted to	Dalal Almatrafi
Submission Date	14,May 2026

Table of Contents

1. Executive Summary
 2. Introduction
 3. Project Requirements Mapping
 4. Secure Database Schema Design
 5. Role-Based Access Control Implementation
 6. AES Encryption for Medical Data
 7. Audit Logging and Accountability
 8. Database Replication for Disaster Recovery
 9. Compliance Analysis with HIPAA Security Rule
 10. Conclusion
- References
- Appendix A: SQL Scripts and Code Evidence

1. Executive Summary

This report presents the design and implementation of the Secure Hospital Management System, a secure Electronic Health Record (EHR) database solution developed to protect sensitive patient information and reduce unauthorized access. The system applies multiple database security mechanisms including Role-Based Access Control (RBAC), AES encryption for sensitive medical records, audit logging, and database replication for disaster recovery. [1].

The proposed design separates patient demographic information from sensitive medical data to enforce the principle of least privilege. Receptionists are limited to basic patient information, while doctors are granted controlled access to medical records without permission to modify billing information. The project also aligns with key HIPAA Security Rule safeguards related to access control, audit controls, integrity, and authentication.

2. Introduction

Hospitals store a combination of demographic information, medical history, diagnoses, prescriptions, billing transactions, and operational records. Because these records can identify a patient and reveal health conditions, they must be handled as highly sensitive information. In the United States regulatory context, electronic protected health information, or ePHI, is subject to the HIPAA Security Rule, which requires technical safeguards such as access control, audit controls, integrity mechanisms, authentication, and transmission security [1].

The objective of this project is to develop a secure EHR database design that protects patient data privacy while allowing authorized hospital staff to perform their duties. The implementation focuses on five required deliverables: secure database schema design, SQL scripts for access control, SQL scripts for encryption, audit logs for patient-data access, database replication for disaster recovery, and a compliance analysis with data privacy laws.

3. Project Requirements Mapping

Requirement	Design Decision	Security Benefit
RBAC for doctors and receptionists	Roles, permissions, role_permissions, MySQL users, grants, and column-level access.	Restricts each user to the minimum data needed for assigned duties.
Doctors cannot modify billing data	Doctor role receives read-only billing access and no INSERT, UPDATE, or DELETE privileges on billing.	Protects financial integrity and prevents unauthorized billing manipulation.
Receptionists see only basic patient details	Receptionist receives SELECT only on patient_id, full_name, phone, and address.	Prevents exposure of medical history, diagnosis, and treatment information.
AES encryption for medical history and diagnoses	Medical fields are stored using AES_ENCRYPT and retrieved with AES_DECRYPT only through authorized access paths.	Reduces plaintext exposure if database rows are viewed or exported.
Audit logs to track patient data access	audit_logs table records user role, action, affected record, table name, timestamp, and IP address.	Supports accountability, investigations, and HIPAA audit-control expectations.
Database replication for disaster recovery	Master-slave metadata tables and replication commands support standby recovery.	Improves availability and reduces downtime during database failure.

4. Secure Database Schema Design

The database schema separates security entities, hospital operational entities, encrypted medical entities, audit entities, and replication metadata. This separation improves maintainability and allows each data category to receive appropriate protection. For example, basic patient contact details are stored in the patients table, while medical history, diagnosis, treatment, and prescriptions are stored in the medical_records table as encrypted binary values. This design is more secure than storing all patient attributes in a single table because it allows strict control over sensitive clinical attributes.

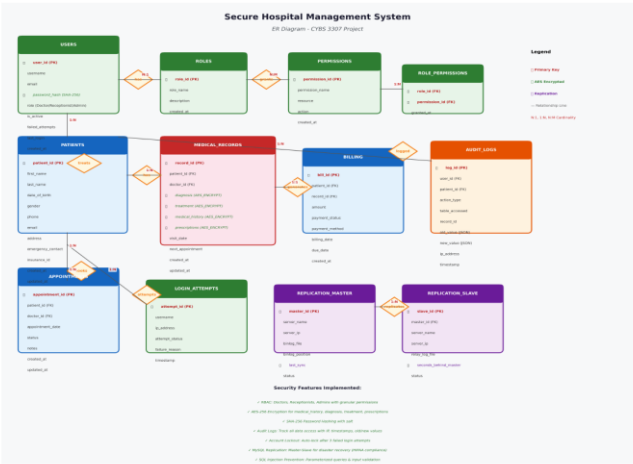


Figure 1. ER Diagram showing secure hospital database schema with RBAC, encrypted medical records, audit logging, and replication entities.

The ER diagram shows a design consisting of twelve major entities: users, roles, permissions, role_permissions, patients, doctors, medical_records, billing, appointments, audit_logs, login_attempts, replication_master, and

replication_slave. The users table connects each account to a role, while role_permissions resolves the many-to-many relationship between roles and permissions. The patients table represents basic patient information, and the medical_records table stores sensitive clinical content in encrypted fields. Billing is separated from medical records to ensure that clinical staff do not automatically receive modification privileges over financial records.

Entity	Purpose	Security Classification
users	Stores system accounts, password hashes, failed attempts, and active status.	Security entity
roles	Defines Doctor, Receptionist, and Admin roles.	Security entity
permissions	Defines resource-action permissions such as patients READ or billing UPDATE.	Security entity
role_permissions	Maps roles to permissions using a composite key.	RBAC junction table
patients	Stores basic demographic and contact information.	Core entity
medical_records	Stores encrypted diagnoses, treatment, history, and prescriptions.	Encrypted sensitive entity
billing	Stores payment and billing status linked to patients and records.	Financial entity
appointments	Stores patient visits and doctor scheduling.	Operational entity
audit_logs	Records access and modification activities.	Audit entity
login_attempts	Tracks successful and failed authentication events.	Security monitoring entity
replication_master	Stores master replication status metadata.	Disaster recovery entity
replication_slave	Stores standby server replication lag and relay-log metadata.	Disaster recovery entity

5. Role-Based Access Control Implementation

Role-Based Access Control is implemented through two complementary layers. The first layer is logical RBAC, where the roles, permissions, and role_permissions tables describe what each user type is allowed to do. The second layer is database-level enforcement, where MySQL users and GRANT statements limit access directly at the database engine. This layered design reduces risk because a user cannot bypass application logic and gain unrestricted database access.

Role	Patient Data	Medical Records	Billing	Appointments
Doctor	Read patient details	Read, create, and update authorized medical records	Read only; no modification	Read assigned appointments
Receptionist	Read basic columns only	No access	No access	Create, read, and update appointments
Admin	Full administrative management	Read encrypted records and manage system operations	Full management	Full management

The requirement that doctors cannot modify billing data is enforced by excluding INSERT, UPDATE, and DELETE billing permissions from the Doctor role. The requirement that receptionists can only see basic patient details is enforced using column-level SELECT access. The screenshot below demonstrates this approach by creating doctor_user and reception_user accounts, then granting the receptionist only selected patient columns.

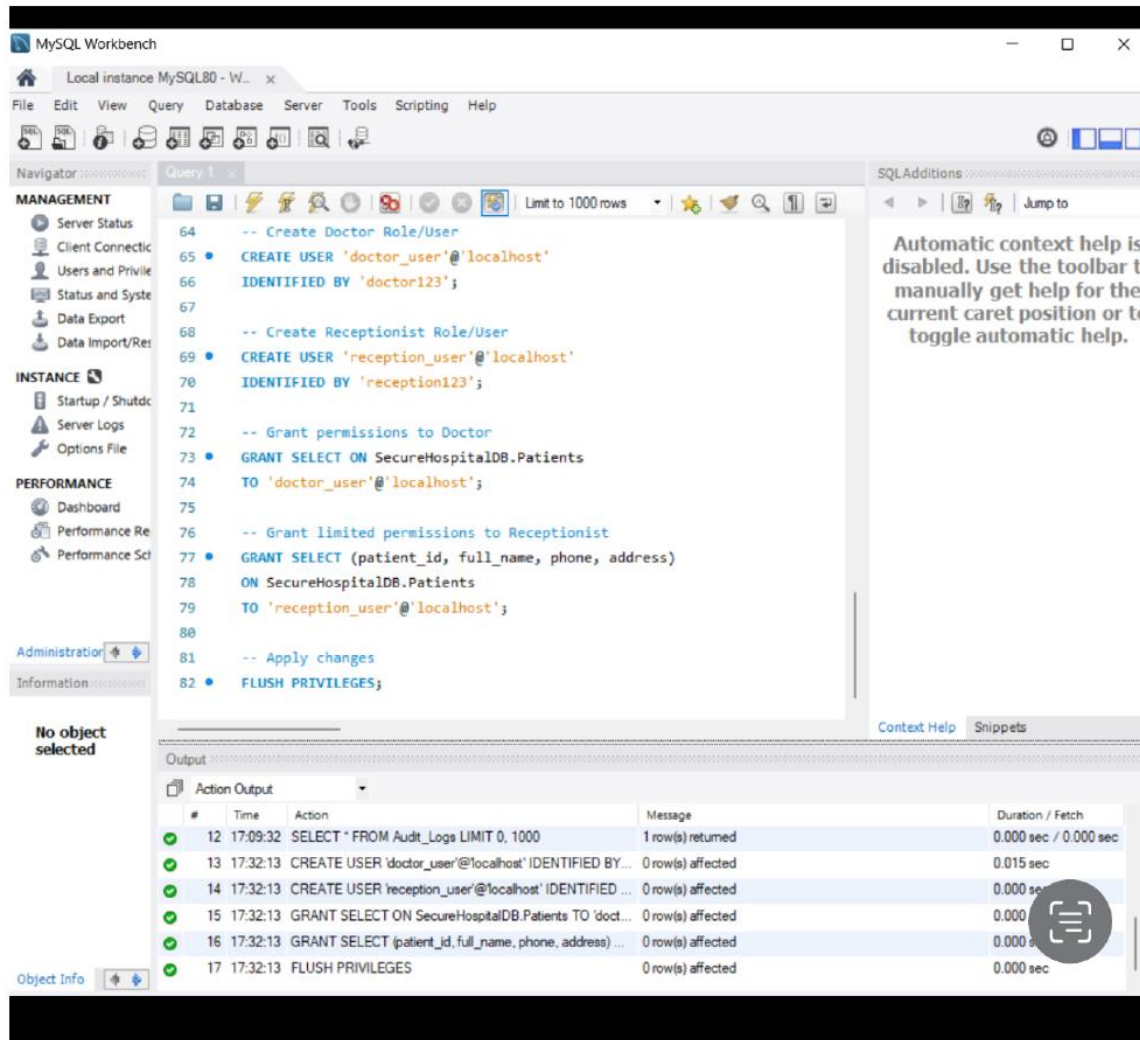


Figure 2. Database-level RBAC using MySQL users and GRANT statements for doctor and receptionist roles.

```

CREATE USER IF NOT EXISTS 'doctor_user'@'localhost' IDENTIFIED BY 'doctor123';
CREATE USER IF NOT EXISTS 'reception_user'@'localhost' IDENTIFIED BY 'reception123';

GRANT SELECT ON SecureHospitalDB.patients TO 'doctor_user'@'localhost';
GRANT SELECT ON SecureHospitalDB.medical_records TO 'doctor_user'@'localhost';
GRANT SELECT ON SecureHospitalDB.billing TO 'doctor_user'@'localhost';

GRANT SELECT (patient_id, full_name, phone, address)
ON SecureHospitalDB.patients TO 'reception_user'@'localhost';
FLUSH PRIVILEGES;

```

6. AES Encryption for Medical Data

AES encryption is used to protect medical history and diagnoses before they are stored in the database. In the implementation screenshots, AES_ENCRYPT is used during patient-data insertion, and AES_DECRYPT is used only when an authorized query retrieves clinical data. HIPAA describes encryption as an addressable implementation specification under access-control and transmission-security requirements, meaning that covered entities must implement it when reasonable and appropriate or document an equivalent alternative [1].

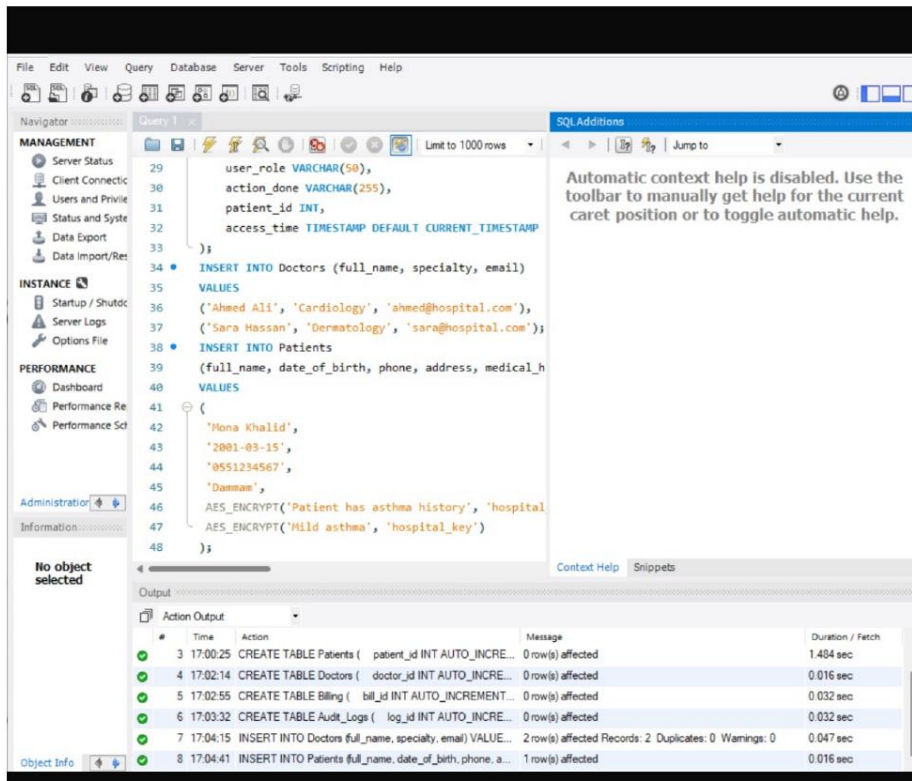


Figure 3. Inserting sample patient and doctor data with AES_ENCRYPT for sensitive medical fields.

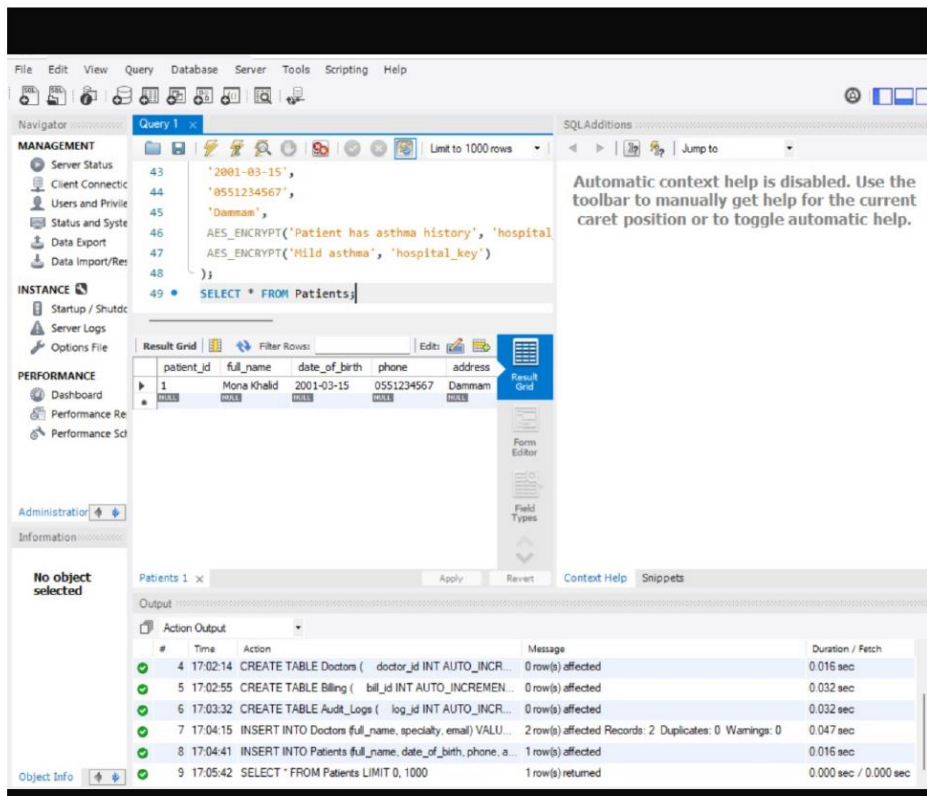


Figure 4. General patient query showing that sensitive medical values are not intended to be exposed as ordinary plaintext.

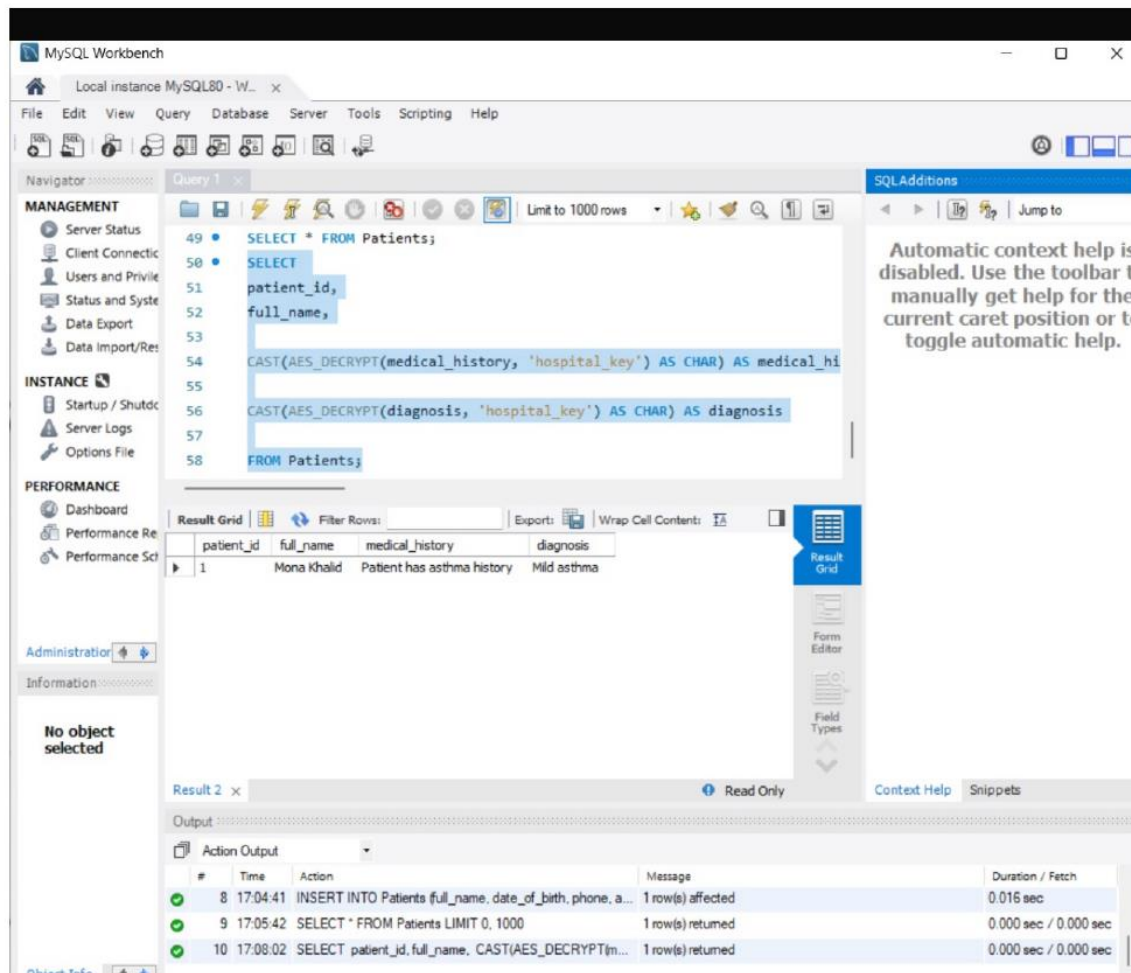


Figure 5. Authorized AES_DECRYPT query used to retrieve medical history and diagnosis for doctor-level access.

In a production deployment, encryption keys should not be hard-coded in SQL scripts. They should be managed through a secure key-management service, database keyring, or hardware security module. The demonstration key in the screenshots is suitable only for academic testing. Key rotation should be scheduled, and historical records should be re-encrypted when a key is replaced.

```

SET @encryption_key = 'hospital_key_32_byte_demo_value';

INSERT INTO medical_records(patient_id, doctor_id, diagnosis, medical_history)
VALUES (
    1,
    1,
    AES_ENCRYPT('Mild asthma', @encryption_key),
    AES_ENCRYPT('Patient has asthma history', @encryption_key)
);

SELECT CAST(AES_DECRYPT(medical_history, @encryption_key) AS CHAR) AS
medical_history,
        CAST(AES_DECRYPT(diagnosis, @encryption_key) AS CHAR) AS diagnosis
FROM medical_records
WHERE patient_id = 1;

```


7. Audit Logging and Accountability

The audit_logs table records patient-data access and database changes. HIPAA technical safeguards require audit controls, meaning covered entities must implement hardware, software, or procedural mechanisms to record and examine activity in systems that contain or use ePHI [1]. The proposed audit log captures the acting user or role, the action performed, the affected table and record, the source IP address, and the timestamp. For update operations, old and new values can be stored as JSON snapshots where appropriate.

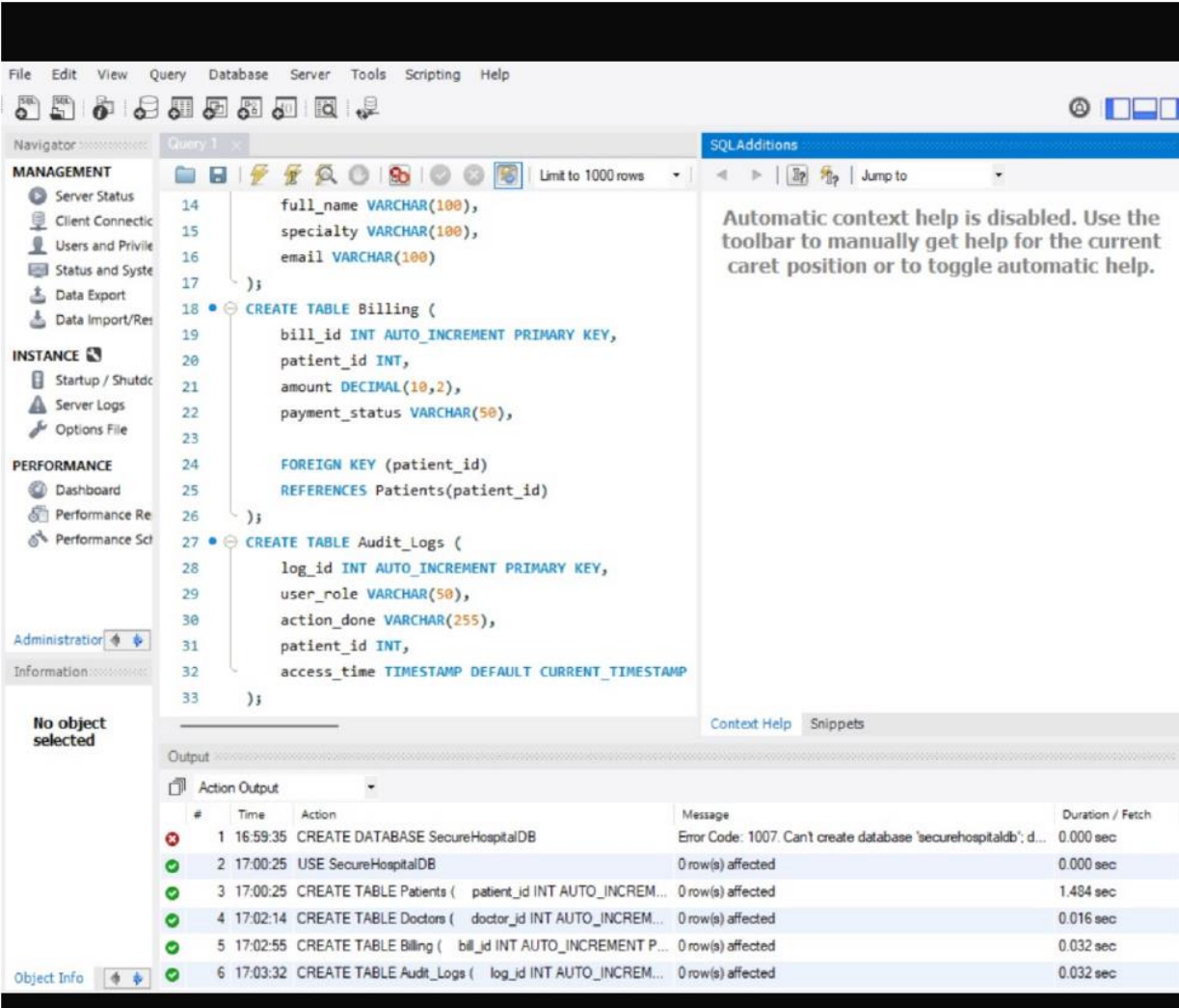


Figure 6. Creating billing and audit-log tables, including timestamped access tracking.

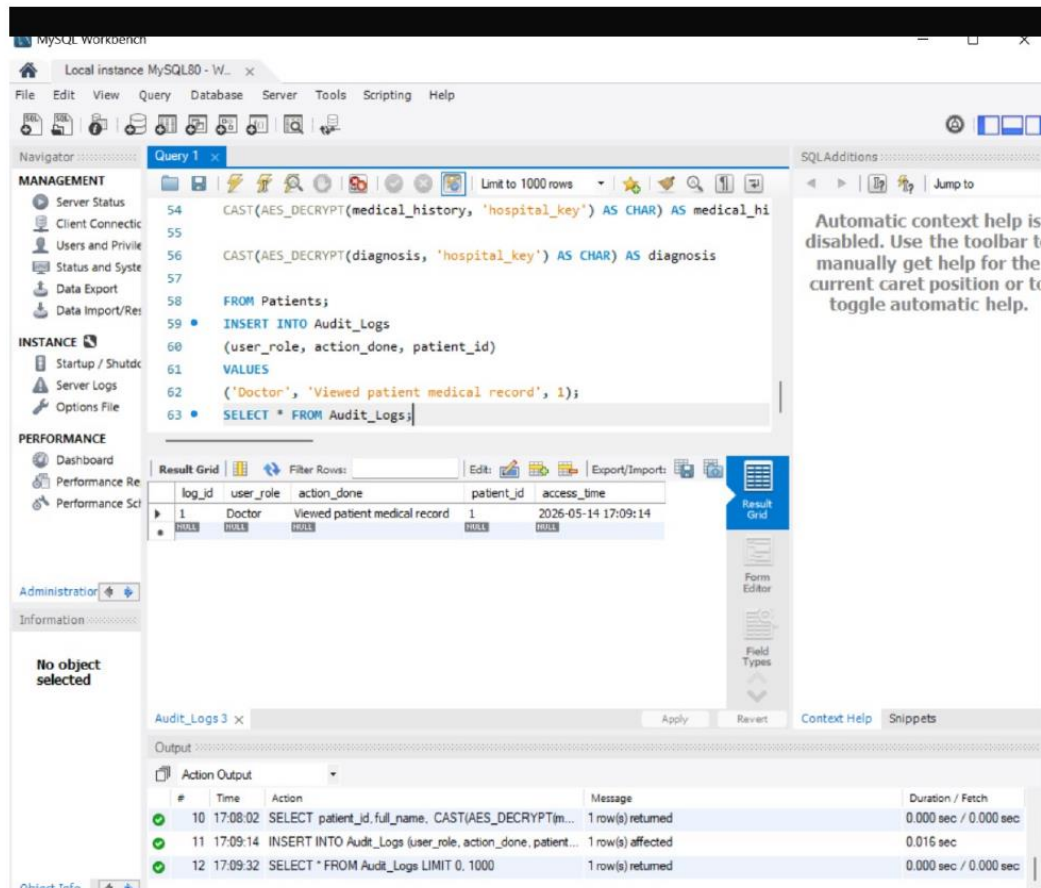


Figure 7. Audit-log record showing that a doctor viewed a patient medical record.

Audit logs should be protected against deletion or unauthorized modification. Administrative access to audit records should be limited, and log retention should follow the organization's legal and institutional requirements. HHS guidance states that Security Rule documentation must generally be retained for six years from creation or last effective date, which supports long-term evidence preservation for security policies and assessments [1].

```

CREATE TABLE audit_logs (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  user_role VARCHAR(50),
  action_done VARCHAR(255) NOT NULL,
  table_accessed VARCHAR(100),
  record_id INT,
  old_value JSON NULL,
  new_value JSON NULL,
  ip_address VARCHAR(45),
  access_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

INSERT INTO audit_logs(user_role, action_done, table_accessed, record_id, ip_address)
VALUES ('Doctor', 'Viewed patient medical record', 'medical_records', 1,
'127.0.0.1');

```

8. Database Replication for Disaster Recovery

Disaster recovery is addressed through database replication. A master database handles write operations and records changes in the binary log, while one or more slave or replica databases receive and replay those changes. Replication improves availability because a replica can be promoted if the master database fails. In this design, replication_master and replication_slave tables store operational metadata such as binary-log file, log position, relay-log file, seconds behind master, and last synchronization time.

Disaster Recovery Component	Implementation	Expected Outcome
Master database	Handles production write transactions and binary logging.	Maintains authoritative copy of hospital records.
Replica database	Receives replicated events and monitors seconds_behind_master.	Provides standby recovery and read-scaling option.
RPO target	Replication lag monitored and alerts triggered if lag exceeds threshold.	Potential data loss kept below the defined target.
RTO target	Failover procedure promotes most current replica and updates application connection.	Hospital operations resume quickly after primary failure.

```
-- On master server
CREATE USER 'replica_user'@'%' IDENTIFIED BY 'strong_replication_password';
GRANT REPLICATION SLAVE ON *.* TO 'replica_user'@'%';
FLUSH PRIVILEGES;
SHOW MASTER STATUS;

-- On slave server
CHANGE MASTER TO
  MASTER_HOST='MASTER_SERVER_IP',
  MASTER_USER='replica_user',
  MASTER_PASSWORD='strong_replication_password',
  MASTER_LOG_FILE='mysql-bin.000001',
  MASTER_LOG_POS=154;
START SLAVE;
SHOW SLAVE STATUS;
```

9. Compliance Analysis with HIPAA Security Rule

The system is designed to support HIPAA Security Rule safeguards for academic and educational purposes rather than claim official HIPAA certification. The Security Rule requires regulated entities to implement technical policies and procedures that allow only authorized persons to access ePHI, record and examine system activity, protect ePHI from improper alteration or destruction, verify user identity, and protect ePHI transmitted over electronic networks [1]. The project design maps each major technical safeguard to a database control as shown below.

HIPAA Security Rule Area	Relevant Requirement	Project Control
Access Control	Allow only authorized persons to access ePHI.	RBAC roles, permissions, database grants, and column-level restrictions.
Audit Controls	Record and examine activity in systems containing ePHI.	audit_logs table, timestamps, user role, action, record ID, and IP address.
Integrity	Protect ePHI from improper alteration or destruction.	Foreign keys, integrity_hash field, controlled updates, and audit triggers.
Authentication	Verify that users are who they claim to be.	users table, password hashes, failed-attempt

		monitoring, and account lockout design.
Transmission Security	Protect ePHI transmitted over electronic networks.	TLS should be enabled for database and application connections in deployment.
Contingency and Availability	Support system recovery and continuity.	Master-replica database design and disaster recovery metadata.

The design also supports privacy principles beyond HIPAA by limiting data collection exposure and separating operational duties. The receptionist does not need access to diagnosis or medical history to schedule appointments; therefore, those fields are excluded from receptionist permissions. Doctors require medical information for care delivery, but they do not require authority to change billing records. This is a direct application of least privilege and separation of duties.

10. Conclusion

The Secure Hospital Management System provides a database-centered security design for protecting EHR data. The schema separates patient demographics, encrypted medical records, billing, appointments, security roles, audit logs, login monitoring, and replication metadata. RBAC prevents doctors from modifying billing data and prevents receptionists from viewing sensitive clinical records. AES encryption protects medical history and diagnosis at rest, and audit logs provide traceability for patient-data access. Database replication supports disaster recovery and continuity of hospital operations.

Overall, the project satisfies the required deliverables: a secure database schema design, SQL scripts for access control and encryption, audit logs for patient-data access, replication planning for disaster recovery, and a compliance analysis mapped to HIPAA technical safeguards. Before production deployment, the system should be enhanced with secure key management, TLS enforcement, multifactor authentication for privileged accounts, automated log monitoring, and formal security-policy documentation.

Future improvements may include multi-factor authentication (MFA), automated security monitoring, and secure cloud-based backup solutions to further strengthen system security.

References

- [1] U.S. Department of Health & Human Services, “Summary of the HIPAA Security Rule.” <https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html>
- [2] U.S. Department of Health & Human Services, “HIPAA for Professionals: Security.” <https://www.hhs.gov/hipaa/for-professionals/security/index.html>
- [3] MySQL Documentation, “AES_ENCRYPT() and AES_DECRYPT() Encryption Functions.” <https://dev.mysql.com/doc/refman/8.0/en/encryption-functions.html>
- [4] MySQL Documentation, “GRANT Statement and Access Control.” <https://dev.mysql.com/doc/refman/8.0/en/grant.html>
- [5] MySQL Documentation, “Replication.” <https://dev.mysql.com/doc/refman/8.0/en/replication.html>

Appendix A: SQL Scripts and Code Evidence

This appendix includes the implementation screenshots supplied for the project. The screenshots show the database creation process, table creation, data insertion, encryption and decryption testing, audit logging, and RBAC grants. A separate SQL file is also provided with the final consolidated script.

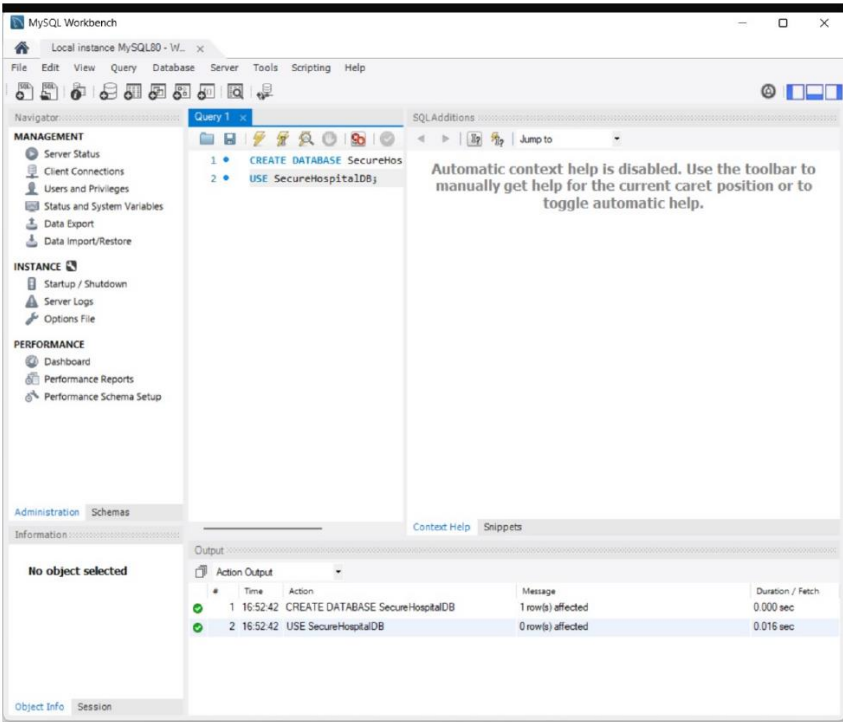


Figure A1. Creating and selecting the SecureHospitalDB database in MySQL Workbench.

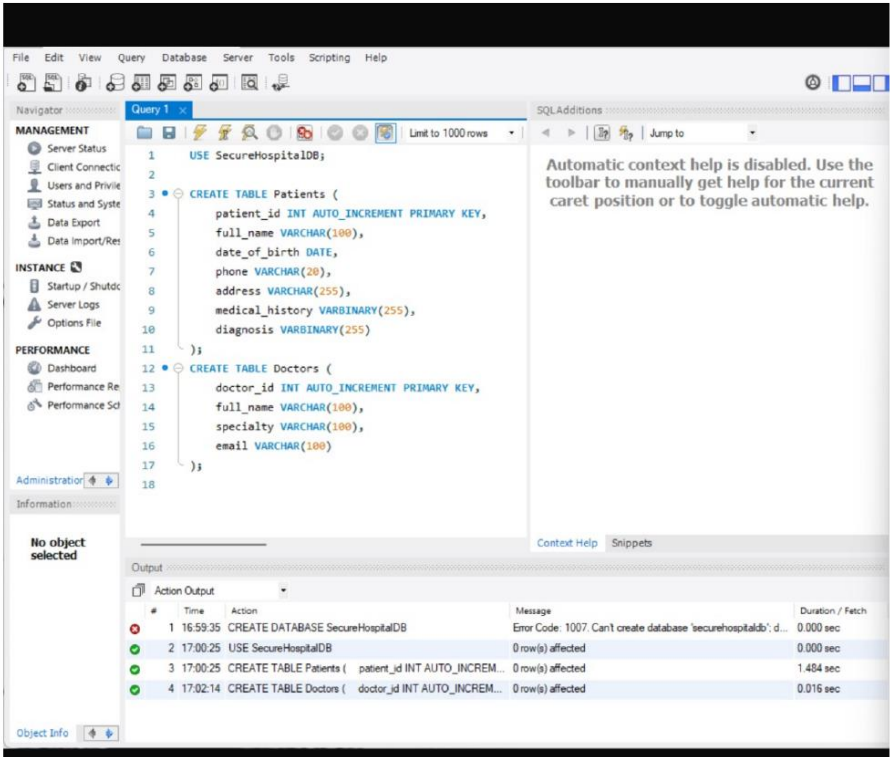


Figure A2. Creating initial Patients and Doctors tables.

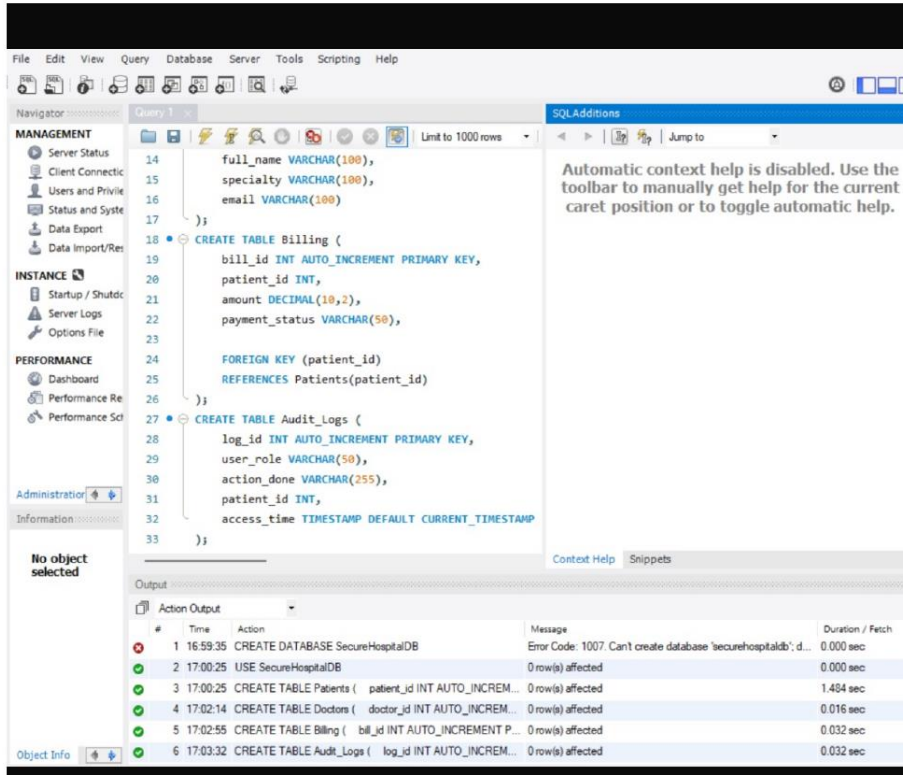


Figure A3. Creating Billing and Audit_Logs tables.

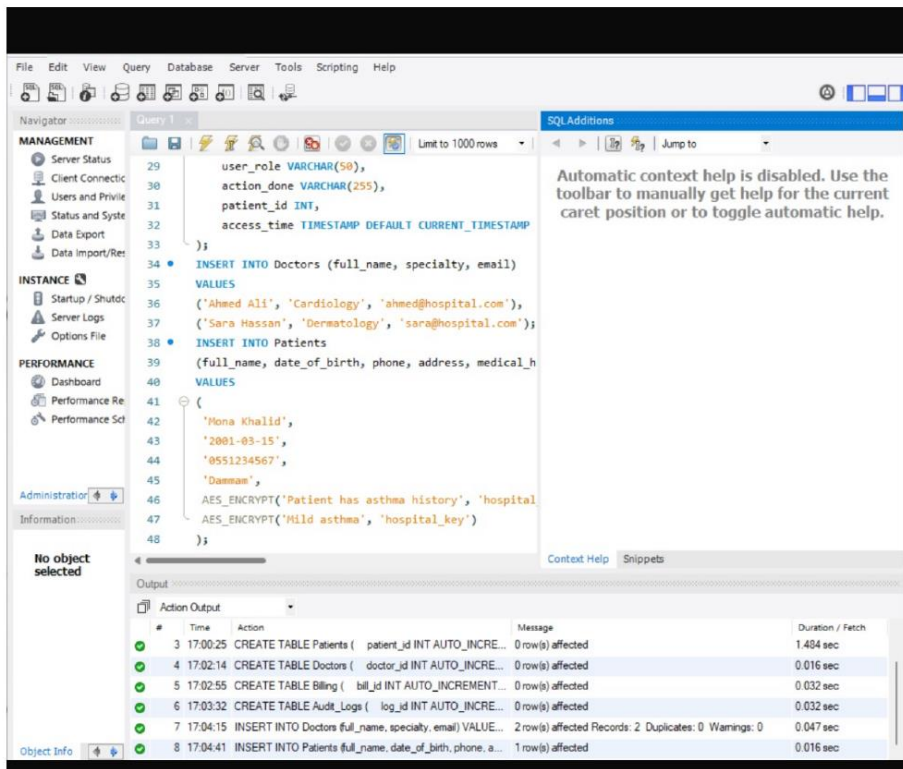


Figure A4. Inserting sample doctors and encrypted patient medical values.

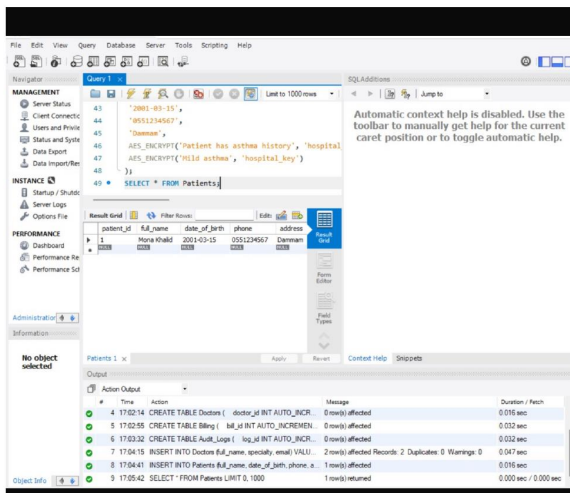


Figure A5. Selecting patient records after insertion.

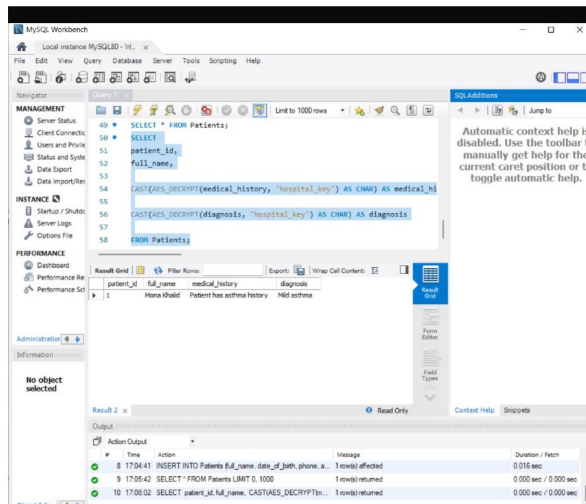


Figure A6. Decrypting medical history and diagnosis using AES_DECRYPT for authorized access.

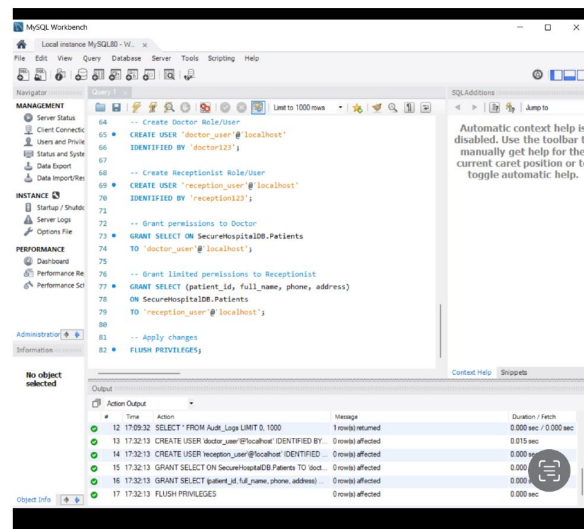
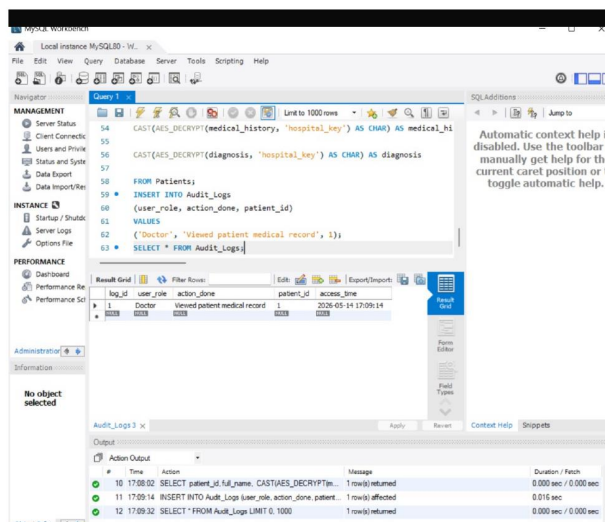


Figure A7. Recording and displaying audit-log activity for a doctor viewing a medical record.

Figure A8. Creating role-based MySQL users and granting doctor/receptionist permissions.